

LeetCode 1004: Max Consecutive Ones III

Problem Notes & Code Implementations

TCS

Sigmoid

Goldman Sachs

Concept & Context

Just like **LC 424**, the only twist is this time we need max consecutive ones, and we are allowed to replace **k** zeroes with 1.

Note: You can refer to notes of LC 424 as the underlying pattern is heavily related.

Key Observation

A sliding window becomes **invalid** when the `zeroCount > k`.

Approach 1: Brute Force

The brute force approach is as simple as generating every subarray and counting the number of zeroes. If zeroes $\leq k$, we update our answer.

Time: $O(N^2)$ | Space: $O(1)$

```
class Solution {
public:
    int longestOnes(vector<int>& nums, int k) {
        int n = nums.size();
        int ans = 0;

        for (int i = 0; i < n; i++) {
            int zeroCount = 0;

            for (int j = i; j < n; j++) {
                if (nums[j] == 0)
                    zeroCount++;

                if (zeroCount <= k)
                    ans = max(ans, j - i + 1);
            }

            return ans;
        }
    };
};
```

Approach 2: Optimized Sliding Window

Maintains a valid window by keeping track of the count of 1s and dynamically adjusting the left pointer when replacements exceed k.

Time: $O(N)$ | Space: $O(1)$

```
class Solution {
public:
    int longestOnes(vector<int>& nums, int k) {
        int n = nums.size();
        int r = 0;
        int l = 0;
        int countone = 0;
        int ans = 0;

        while (r < n) {
            if (nums[r] == 1) countone++;

            while ((r - l + 1) - countone > k) {
                if (nums[l] == 1) countone--;
                l++;
            }

            ans = max(ans, r - l + 1);
            r++;
        }

        return ans;
    }
};
```